

Real-Time Groundwater Resource Evaluation Using DWLR Data

Period: June 1, 2026 – June 13, 2026

Days: Day 1 to Day 13 (13 Working Days)

Phase: Planning · Design · Environment Setup

Tech Stack: React.js · Spring Boot · MySQL · DWLR API

This document records the day-by-day planned activities, tasks performed, and expected deliverables for the mini project spanning June 1 to June 13, 2026. Actual diagrams are embedded on Day 5 (System Flow Diagram) and Day 7 (Groundwater Resource Data-Flow Diagram).

Table of Contents

Day	Date	Activity
Day 1	June 1, 2026	Project Title Finalization
Day 2	June 2, 2026	Requirement Gathering
Day 3	June 3, 2026	Objective Definition
Day 4	June 4, 2026	User & Module Identification
Day 5	June 5, 2026	Use Case Diagram Preparation
Day 6	June 6, 2026	Database Requirement Analysis
Day 7	June 7, 2026	ER Diagram Design
Day 8	June 8, 2026	Database Schema Creation
Day 9	June 9, 2026	UI Wireframe Design
Day 10	June 10, 2026	Login & Dashboard UI Design
Day 11	June 11, 2026	Navigation & Form Design
Day 12	June 12, 2026	Design Review
Day 13	June 13, 2026	Frontend Environment Setup

Overview

On the first day of the mini project, the team focuses on finalizing a clear, concise, and technically appropriate project title. The title **"Real-Time Groundwater Resource Evaluation Using DWLR Data"** was proposed and reviewed. DWLR (Digital Water Level Recorder) sensors provide automated, continuous water-level readings from bore wells, making them ideal for real-time groundwater evaluation.

Key Tasks

1.	Review scope of groundwater monitoring and DWLR technology.
2.	Brainstorm and shortlist potential project titles.
3.	Evaluate titles against feasibility, relevance, and uniqueness.
4.	Obtain mentor/guide approval for the finalized title.

Outcome

The project title is officially approved: *Real-Time Groundwater Resource Evaluation Using DWLR Data*. This sets the foundation and direction for all subsequent work.

Overview

Requirement gathering involves identifying the core problem the project aims to solve. Groundwater depletion is a critical challenge in India, particularly in Tamil Nadu. Traditional manual monitoring is infrequent and error-prone. DWLR sensors capture water-level data at regular intervals, but efficient storage, retrieval, and visualization of this data remains a gap.

Key Tasks

1.	Identify stakeholders: hydrogeologists, farmers, government agencies.
2.	Document current pain points in manual groundwater monitoring.
3.	Define functional requirements: data ingestion, storage, dashboard, alerts.
4.	Define non-functional requirements: real-time response, scalability, security.
5.	Draft the formal Problem Statement document.

Outcome

A structured Problem Statement is prepared, articulating that manual groundwater monitoring is insufficient for real-time decision-making and that a DWLR-integrated digital system is required.

Overview

With the problem clearly stated, the team defines measurable and specific objectives that the system must achieve. Objectives serve as success criteria throughout development and evaluation phases.

Key Tasks

1.	Translate problem requirements into SMART objectives.
2.	Categorize objectives: primary (core goals) and secondary (enhancements).
3.	Align objectives with available DWLR datasets and technology stack.
4.	Document and validate objectives with the project guide.

Outcome

Finalized project objectives: (1) Collect and store real-time DWLR water-level data; (2) Provide a web-based dashboard for visualization; (3) Generate alerts for critical groundwater levels; (4) Support trend analysis and historical comparison; (5) Enable role-based access for stakeholders.

Overview

This day focuses on identifying the different categories of users who will interact with the system and breaking down the system into logical modules. Clear user roles and module boundaries prevent scope creep and guide the database and UI design.

Key Tasks

1.	Identify user roles: Admin, Field Engineer, Government Official, Public User.
2.	Define access levels and permissions per role.
3.	Break the system into modules: User Management, DWLR Data Ingestion, Groundwater Analytics, Alert Management, Report Generation, GIS Mapping.
4.	Document the Module List with brief descriptions.

Outcome

A comprehensive Module List is produced covering six core modules. Each module has a defined owner, input/output specification, and associated user roles.

Overview

A Use Case Diagram provides a high-level visual representation of how different actors interact with the system. For this project, actors include the Admin, Field Engineer, Government Official, and the DWLR Sensor (as an external system). Use cases cover login, data upload, dashboard view, threshold alerts, and report export. The flowchart below illustrates the complete system data-flow and decision logic for DWLR-based groundwater evaluation.

Key Tasks

1.	List all actors and their primary interactions.
2.	Map each actor to relevant use cases.
3.	Define include and extend relationships between use cases.
4.	Draw the Use Case / Flow Diagram using draw.io or StarUML.
5.	Review and submit the diagram.

Outcome

A complete system flow diagram is prepared and submitted, illustrating all major interactions including network configuration, path initialization, sink node movement, relay strategy, and combining strategy.

Diagram

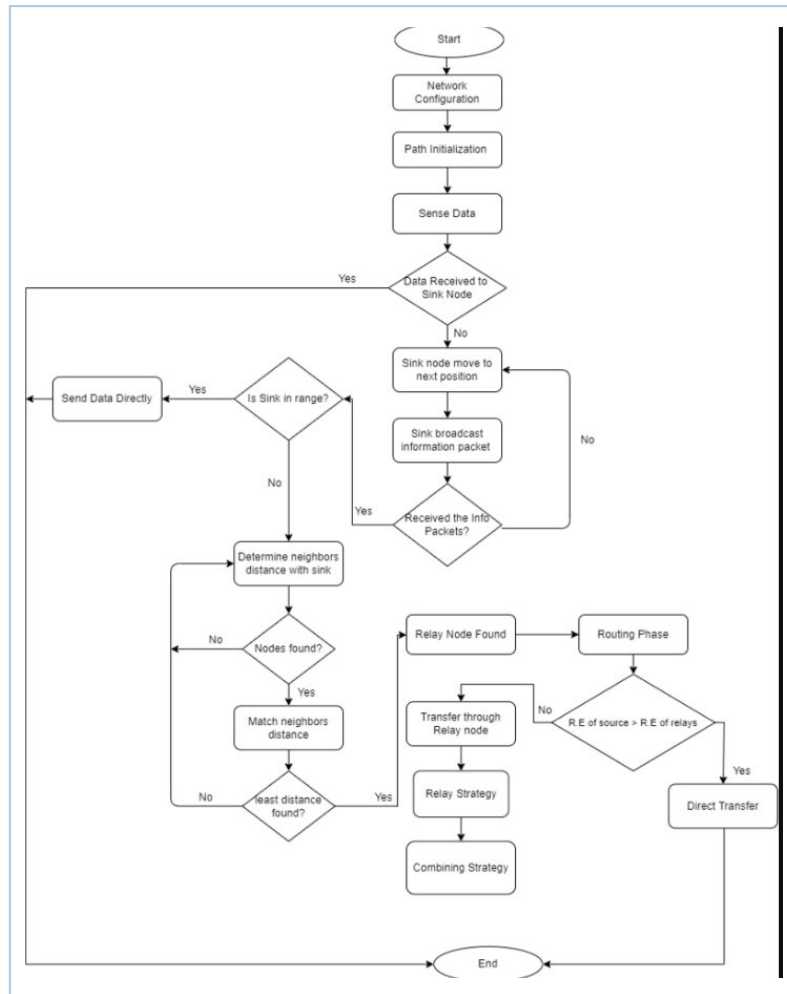


Figure 1: System Flow Diagram – DWLR Data Routing & Relay Strategy

Overview

Database Requirement Analysis involves identifying all the data entities the system must store, their attributes, and the relationships between them. For a DWLR-based system, key entities include sensors, wells, readings, users, alerts, and reports.

Key Tasks

1.	Enumerate all data entities from the requirements and module list.
2.	Identify primary attributes for each entity.
3.	Determine cardinality of relationships (one-to-many, many-to-many, etc.).
4.	List tables with proposed column names and data types.
5.	Validate completeness against all functional requirements.

Outcome

A finalized Table List with 8 primary tables: Users, Roles, Wells, DWLR_Sensors, Water_Level_Readings, Alerts, Reports, and Locations.

Overview

The Entity-Relationship (ER) Diagram visually models the database structure by depicting entities, their attributes, and the relationships between them. This is a critical artifact that directly translates into the database schema. The diagram below shows the groundwater resource mapping workflow — from geological and bibliographic input data through to the final Groundwater Resource Map.

Key Tasks

1.	Create entities based on the Table List from Day 6.
2.	Define primary keys (PK) and foreign keys (FK) for each entity.
3.	Draw relationships with cardinality notations.
4.	Include weak entities where applicable (e.g., Readings linked to Sensors).
5.	Review and finalize ER Diagram using draw.io or MySQL Workbench.

Outcome

A complete ER / data-flow diagram is produced showing all entities with their attributes and relationships clearly annotated, including ISPRA Legend Guidelines, Field Survey data, Climatic Data, and Portofino Park DB inputs.

Diagram

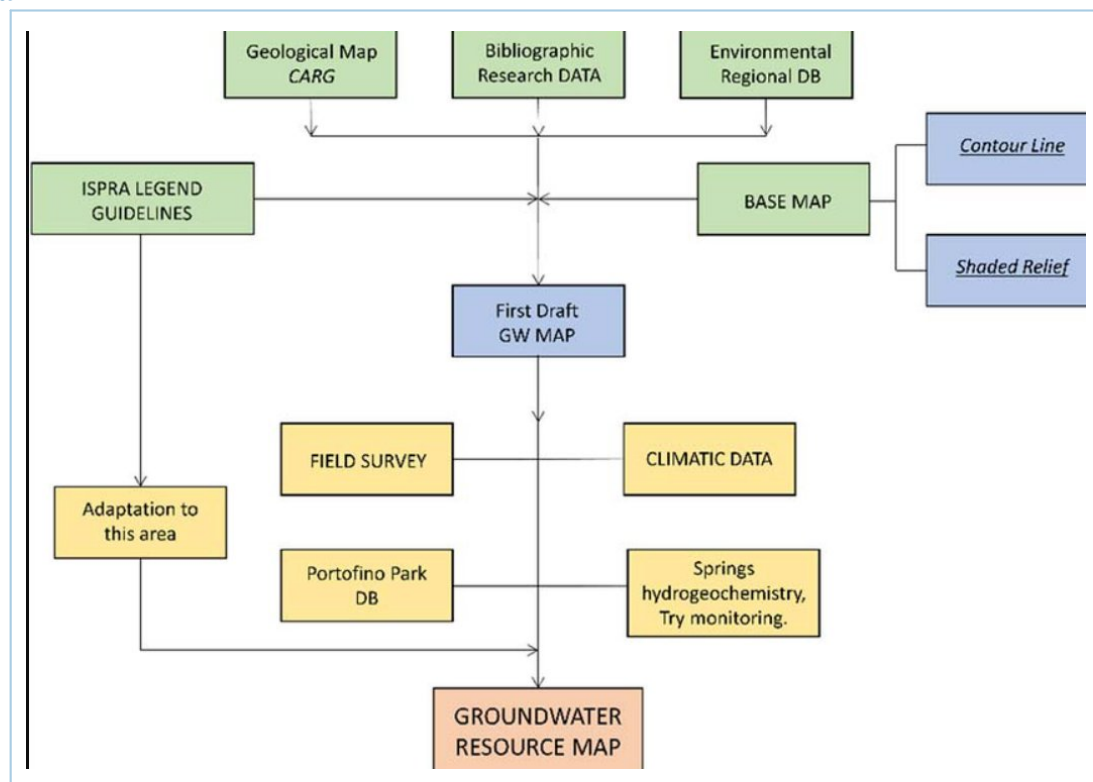


Figure 2: Groundwater Resource Evaluation Data-Flow Diagram

Overview

Based on the ER Diagram, the SQL schema is written — defining CREATE TABLE statements with proper data types, constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE), and indexes. This forms the backbone of the backend data layer.

Key Tasks

1.	Translate each ER entity into a SQL CREATE TABLE statement.
2.	Apply normalization rules (at least 3NF) to eliminate redundancy.
3.	Define constraints: PK, FK, NOT NULL, DEFAULT, CHECK.
4.	Create indexes on frequently queried columns (e.g., sensor_id, timestamp).
5.	Test schema creation in MySQL/PostgreSQL locally.

Outcome

A fully tested SQL schema file is produced and documented. The schema includes all 8 tables with proper constraints and foreign key relationships.

Overview

UI Wireframing involves sketching the page layout and structure of each screen in the application before actual development begins. Wireframes are low-fidelity representations that focus on layout, navigation flow, and content placement.

Key Tasks

1.	List all required pages: Login, Dashboard, Well Map, Data Table, Alerts, Reports.
2.	Sketch wireframes for each page using Figma or Balsamiq.
3.	Define navigation flow between pages.
4.	Incorporate DWLR-specific components: water level gauge, trend chart, GIS map.
5.	Review wireframes with the project team and guide.

Outcome

Page layout wireframes for 6 core screens are completed, showing content hierarchy, navigation elements, and key interactive components.

Overview

Moving from wireframes to high-fidelity UI designs, this day focuses on designing the Login Page and the main Dashboard — the central hub displaying real-time DWLR sensor readings, water level trends, alerts, and a map view of monitored wells.

Key Tasks

1.	Design the Login page with email/password fields, forgot-password link.
2.	Design the Dashboard layout: KPI cards (total wells, active sensors, alerts).
3.	Add real-time water level charts (line charts) using Recharts/Chart.js mock.
4.	Include a map component placeholder for GIS well locations.
5.	Apply a consistent color theme for an environmental monitoring app.

Outcome

High-fidelity UI screen designs for the Login Page and Dashboard are completed in Figma, ready for frontend development.

Overview

This day extends the UI design to include the navigation structure (sidebar/topbar) and all data entry forms. Forms are critical for sensor registration, manual data entry, and threshold alert configuration.

Key Tasks

1.	Design the sidebar navigation with icons for each module.
2.	Create form designs: Add Well, Register Sensor, Set Alert Threshold, Add User.
3.	Incorporate validation states: error, success, and loading indicators.
4.	Link all designed screens into a clickable prototype in Figma.
5.	Conduct a walkthrough of the prototype with the project guide.

Outcome

A clickable UI Prototype covering all primary screens is delivered, enabling stakeholder review before development begins.

Overview

The Design Review day is dedicated to a structured review of all design artifacts produced so far — ER Diagram, SQL Schema, Wireframes, and UI Prototype. Feedback is gathered from the project guide and peers, and necessary revisions are made before development begins.

Key Tasks

1.	Present ER Diagram, SQL Schema, and UI Prototype to the reviewer.
2.	Document all feedback and requested changes.
3.	Revise UI screens and wireframes based on feedback.
4.	Verify schema completeness against all functional requirements.
5.	Obtain formal sign-off / design approval.

Outcome

All design artifacts are reviewed, revised, and officially approved. A Design Approval document is signed off, clearing the project for frontend development.

Overview

With designs approved, the development phase begins. Day 13 focuses on setting up the frontend development environment using React.js with a clean project scaffold configured with all necessary dependencies, folder structure, routing, and state management tools.

Key Tasks

1.	Initialize the React project using Create React App or Vite.
2.	Install core dependencies: React Router, Axios, Recharts, Tailwind CSS / MUI.
3.	Set up project folder structure: /pages, /components, /services, /hooks, /assets.
4.	Configure React Router with initial route placeholders.
5.	Set up environment variables for API base URL.
6.	Push the initial project scaffold to the GitHub repository.

Outcome

A fully configured React project is created, pushed to GitHub, and verified to run successfully on localhost. The development environment is ready for feature implementation starting Day 14.